CHALLENGE | ONG TURMA DO BEM

Raiz do Bem - Plataforma integrada para a Ong Turma do Bem

Documentação – Sprint 3: Domain Driven Design Using Java

MURILO AYABE SEVERINO - RM567479

PAULO CAVALCANTE CAROBA - RM566667

RENAN DA SILVA PAULINO - RM566610

Tecnólogo em Análise e Desenvolvimento de Sistemas

1TDSPS – Prof. Fernando Luiz

SÃO PAULO

2026

Sumário

Introdução 1
Descrição geral do projeto 1
Objetivo 1
Justificativa da Solução 1
Funcionalidades implementadas 2
Visualização de telas do sistema 4
Métodos com lógica de negócio 7
Modelo de Entidade Relacionamento – (MER) 10
Arquitetura do Projeto 11
Diagramas de Classes 12
Diagrama de Classes Simples 12
Diagrama UML 13
Procedimentos para execução 14
Demonstração de funcionamento do fluxo 18
Execução do main com fluxo central 22
Referências e links relevantes 23
Conclusão 24

Total de Páginas: 27

INTRODUÇÃO

A Ong Turma do Bem foi fundada inspirada pelo trabalho de um dentista, autor do livro "Um Sorriso Feliz para o Seu Filho". Durante palestras em escolas, percebeu-se que as famílias de escolas particulares conseguiam aplicar os conselhos de saúde bucal, enquanto em escolas públicas os relatos eram de problemas odontológicos graves e sem condições de resolução. Assim, nasceu uma organização social dedicada a transformar a vida de jovens de baixa renda por meio do acesso gratuito à tratamento odontológico.

Hoje, a Turma do Bem conta com aproximadamente 18,5 mil dentistas voluntários e 320 coordenadores, tendo beneficiado cerca de 90 mil pessoas. Com atuação em 120 municípios brasileiros, a ONG também está presente em 12 países da América Latina e em Portugal, sempre dependente do apoio de patrocinadores para manter suas atividades.

Atualmente, o atendimento e a gestão dos casos ocorrem de forma descentralizada, empregando canais como WhatsApp, e-mail e formulários simples, o que resulta em falhas, atrasos e perda de informações. Diante desse contexto, este documento apresenta a primeira entrega do projeto Raiz do Bem: uma plataforma digital desenvolvida com base nos conceitos de Domain Driven Design, que visa centralizar o atendimento, otimizar fluxos e trazer mais transparência e organização para os processos da ONG Turma do Bem.

DESCRIÇÃO GERAL DO PROJETO

Objetivo

O sistema "Raiz do Bem" foi desenvolvido para gerenciar atendimentos odontológicos prestados por uma ONG, permitindo o cadastro e acompanhamento de beneficiários, dentistas, pedidos de ajuda e atendimentos realizados.

Justificativa da Solução

Considerando os desafios operacionais da ONG Turma do Bem — como informações distribuídas em diferentes canais, processos manuais e baixa rastreabilidade dos atendimentos — a solução proposta evoluiu para uma plataforma estruturada em Java com foco em organização, persistência e confiabilidade dos dados.

Na Sprint 3, o projeto consolidou a arquitetura em camadas (View, Controller, BO e DAO), com integração efetiva ao banco Oracle, permitindo que os processos essenciais tenham persistência real e fluxo consistente de ponta a ponta. A implementação de operações CRUD nos módulos centrais trouxe maior controle sobre o ciclo de vida dos dados, reduziu retrabalho e melhorou a qualidade das informações utilizadas nas decisões da ONG.

A modelagem orientada ao domínio se justifica pela natureza sensível do problema social e de saúde atendido: ela permite representar regras de negócio com maior precisão, separar responsabilidades técnicas e preparar o sistema para evolução contínua sem perda de consistência. Além disso, a adoção de entidades de apoio com catálogo fixo (como especialidades e programas sociais) reforça padronização e integridade no cadastro, evitando inconsistências operacionais.

Com essa evolução, a solução atende melhor aos objetivos de eficiência, transparência e governança, criando uma base tecnológica mais robusta para auditoria, geração de métricas e expansão futura. Dessa forma, o projeto contribui para profissionalizar a gestão da ONG, ampliar seu impacto social e fortalecer a prestação de contas para patrocinadores, voluntários e sociedade.

FUNCIONALIDADES IMPLEMENTADAS

Endereço

- Criar endereço com validação da API ViaCep e regex.
- Listar todos os endereços
- Buscar endereço por ID
- Filtrar por cidade (lógica de negócio)
- Atualizar endereço – também conta com validação da API ViaCep
- Excluir endereço

Dentista

- Criar dentista com validação de sexo/categoria
- Listar todos os dentistas
- Filtrar dentistas por cidade (lógica: relação com Endereço)
- Listar apenas os dentistas disponíveis (lógica: filtro booleano)
- Buscar dentista por CPF
- Atualizar dados do dentista
- Excluir dentista

Colaborador

- Criar colaborador
- Listar todos os colaboradores
- Buscar por CPF
- Atualizar dados do colaborador
- Excluir colaborador

Pedido de Ajuda

- Criar pedido com validação de CPF com regex.
- Listar todos os pedidos
- Buscar por CPF
- Buscar por ID
- Filtrar pedidos por data específica
- Atualizar status e atribuir dentista, para permitir aprovação/rejeição
- Excluir pedido

Beneficiário

- Criar beneficiário - recebe os dados inseridos no Pedido Ajuda
- Listar todos os beneficiários
- Filtrar beneficiários por programa social (lógica: relação com Programa Social)
- Filtrar beneficiários por cidade (lógica: relação com Endereço)
- Buscar por CPF
- Atualizar dados
- Excluir beneficiário

Atendimento

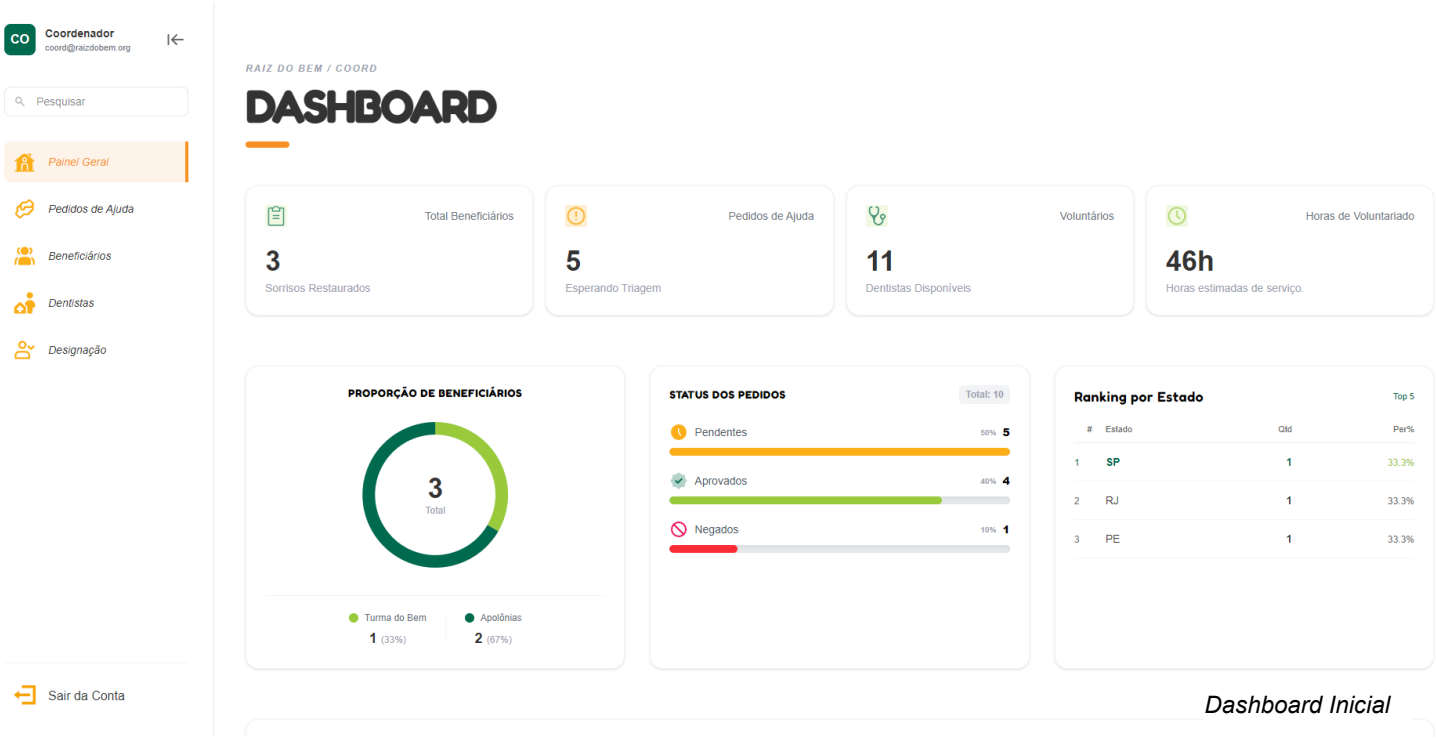
- Criar atendimento
- Listar todos os atendimentos
- Buscar por CPF do beneficiário - JOIN com Beneficiário
- Busca por ID
- Finalizar atendimento – atualizando dados com prontuário, colaborador responsável que relata, e atualizando data final.

Módulos de Apoio

- Listar catálogos fixos
- Validar existência por ID para uso em fluxos de negócio
- Sem CRUD completo por decisão de domínio (dados de referência)

VISUALIZAÇÃO DE TELAS DO SISTEMA

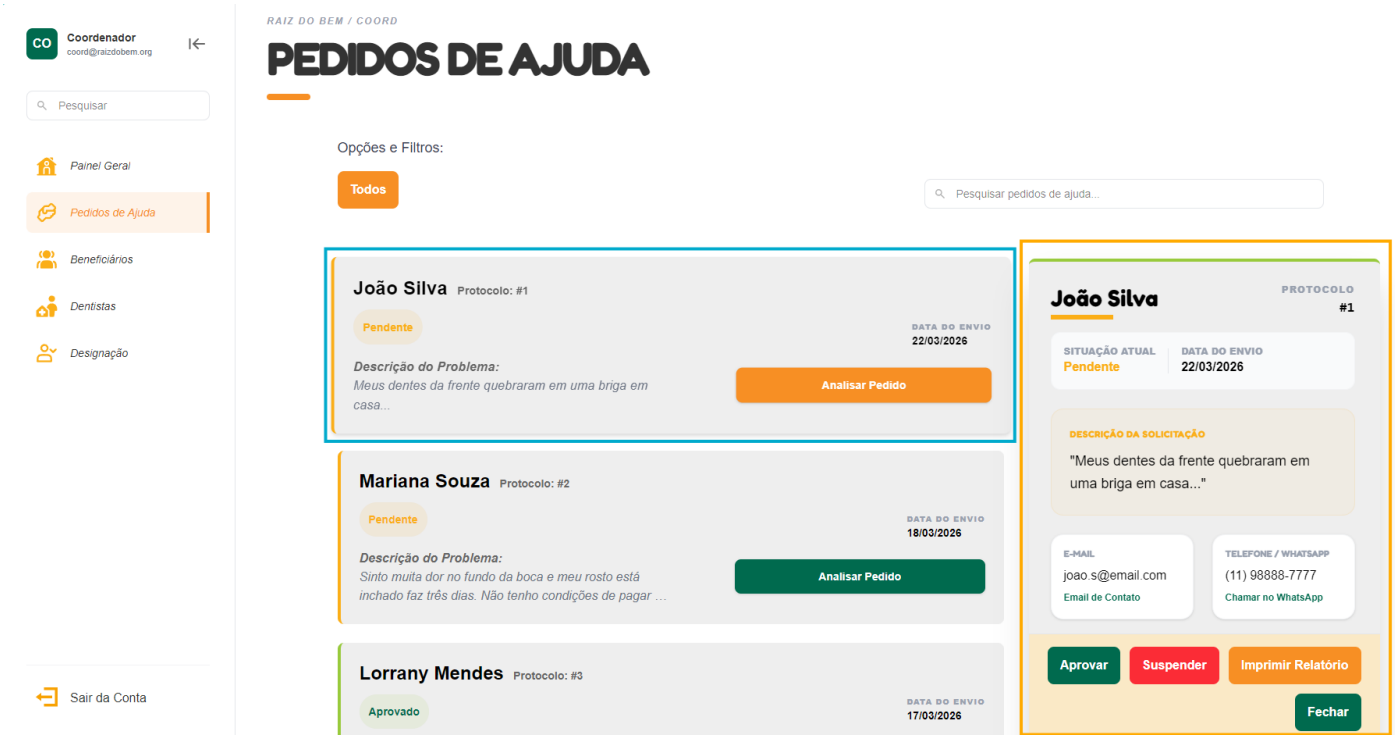
Painel Geral



Dashboard Inicial

Tela inicial do nosso sistema, com uma visão geral, que nos permite analisar informações sobre beneficiários, pedidos de ajuda, voluntários, e gráficos da ONG. É possível notar, como o site conata com uma boa identidade visual, intuitiva e de fácil uso.

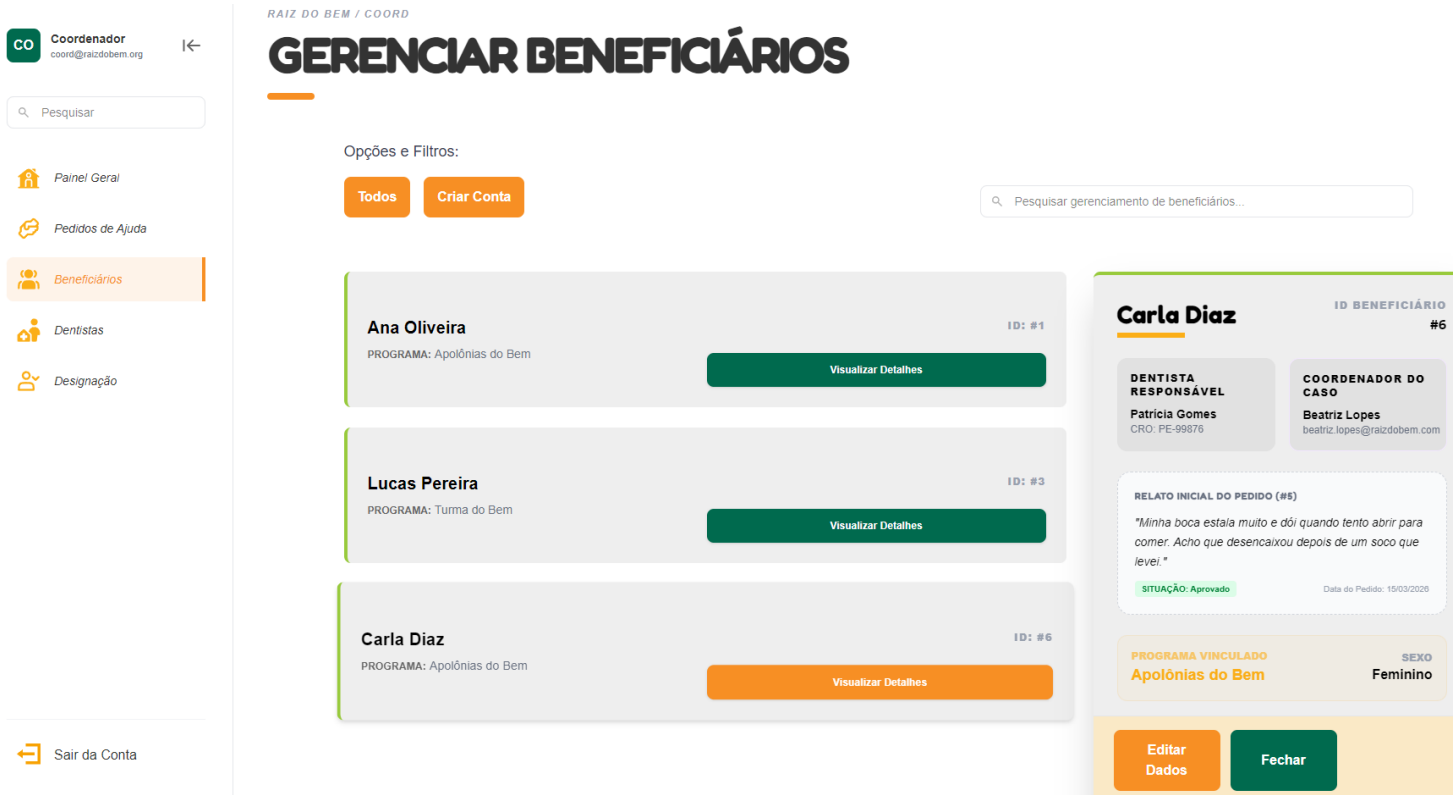
Pedidos de Ajuda



Análise de Pedido de Ajuda

Nessa segunda tela, já implementada, é possível visualizar todos os pedidos de ajuda encontrados. Clicando no botão de analisar pedido, aparecem as possibilidades de Aprovar/Rejeitar o pedido de ajuda, e a opção de imprimir um relatório desse pedido de ajuda.

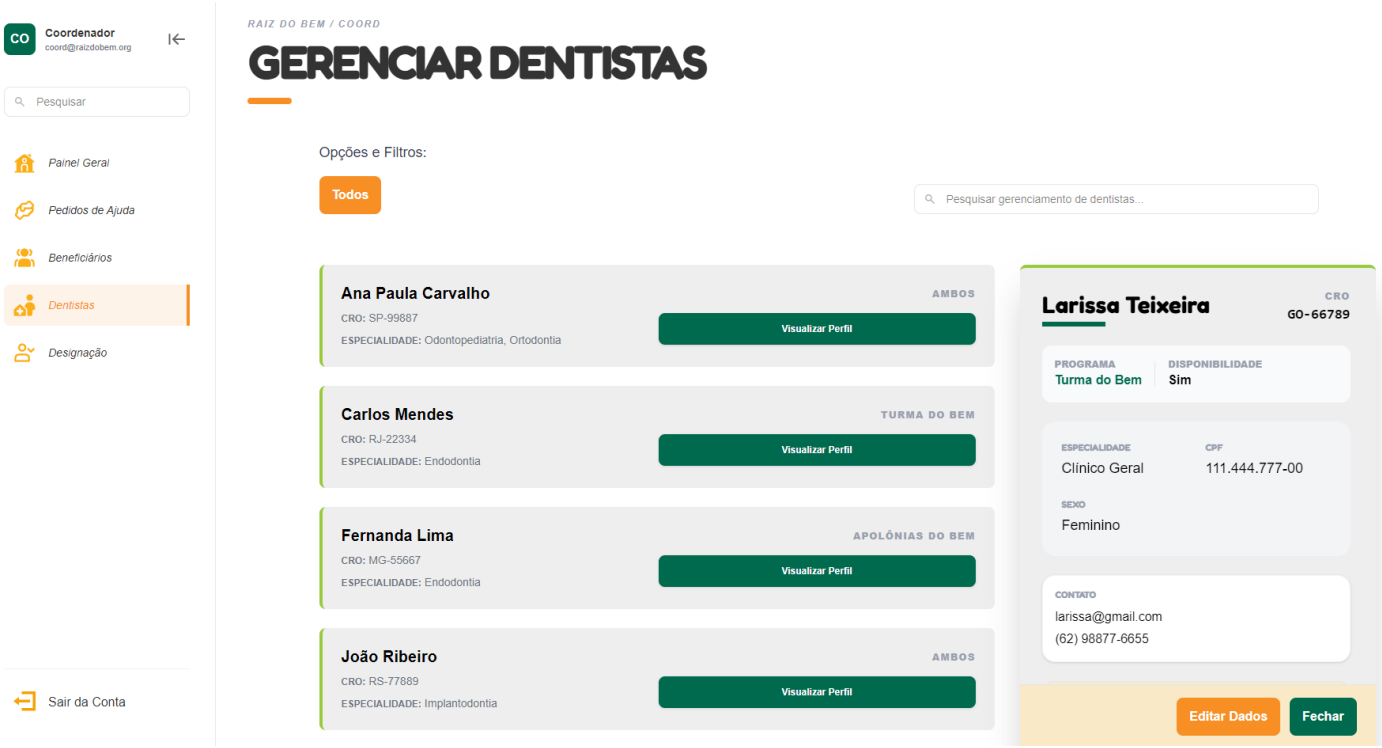
Beneficiários



Verificação geral dos beneficiários

Nessa tela de beneficiários, pode se ver, todas as informações do beneficiário que está no sistema da ONG. Incluindo informações sobre o seu atendimento e dentista responsável pelo caso.

Dentistas



Informações gerais dos dentistas

Nesta seção, são exibidas todas as informações sobre os dentistas incluindo seus dados, especialidades, e informações sobre o seu local de atendimento.

MÉTODOS COM LÓGICA DE NEGÓCIO

1. Método validarEndereco -> EnderecoBO

```java
public Endereco validarEndereco(String cep, String numero, TipoEndereco tipoEndereco){
        ViaCep enderecoBuscado = bo.buscarInformacoesEndereco(cep);

        if(enderecoBuscado == null || enderecoBuscado.isErro()){
            throw new RuntimeException("Endereço não encontrado!!!");
        }
        return new Endereco(
                enderecoBuscado.getLogradouro(),
                cep,
                numero,
                enderecoBuscado.getBairro(),
                enderecoBuscado.getLocalidade(),
                enderecoBuscado.getUf(),
                tipoEndereco
        );
    }
```

Trecho de código do validarEndereco

Valida o cep do endereço, com integração com API do ViaCep. Após digitação do usuário, o método busca pelo cep, e se for válido, são preenchidas todas as informações de logradouro, bairro, cidade e estado, faltando ao usuário responder apenas o número do local, e o tipo, se é residencial ou profissional, o que economiza tempo na experiência do usuário.

2. Método validarCpf -> Validação

```java
public boolean validarCpf(String cpf) {
        return cpf != null && cpf.matches("\\d{11}");
    }
```

Validação do Cpf

Esse método, criado para validar CPF, garante que o valor inserido será atendido em caso específico, e todas as classes que a utilizem terão garantidos 11 caracteres numéricos na inserção do campo para validação firme, já que conta com uso de regex. Garante que em caso de digitação incorreta, é necessário reinserir

dado. Método reutilizável, que foi implementado em uma classe geral, para permitir o reuso por outras classes.

3. **Método validarCep -> EnderecoBO**

```
public boolean validarCep(String cep){
        return cep!=null && cep.matches("\\d{8}");
    }
```

Valida entrada do Cep

Método semelhante ao de validação de CEP, neste caso, também é utilizado o regex, para validação de campo, numérico, e neste caso, com 8 dígitos, padrão brasileiro para o documento de CEP. Este método permite nova digitação em inputs, caso seja digitado incorretamente, garantindo a entrada de dados desse tipo.

4. **Método validarGerarBeneficiario -> PedidoAjudaBO**

```
public void validarGerarBeneficiario(int id, PedidoAjuda novoPedido, int idPrograma){
        PedidoAjuda pedidoAjuda = dao.buscarPorId(id);

        if(pedidoAjuda == null){
            throw new RuntimeException("Pedido não encontrado!!!");
        }
        dao.atualizarPedido(id, novoPedido);

        BeneficiarioBO bo = new BeneficiarioBO();
        bo.adicionar(pedidoAjuda.getIdPedido(), idPrograma);
    }
```

Atualiza o status e gera beneficiário automaticamente

Com este quarto método de regra de negócio, o validarGerarBeneficiario, é possível meio que automatizar a criação de um beneficiário, onde, caso um pedido de ajuda seja atualizado com o status de “APROVADO”, esse método, é implementado, e reduz o processo manual de criação, antes em dois passos – atualização de pedido de ajuda + criação de novo beneficiário, - agora, conta com o preenchimento automático de informações capturadas previamente. Validação essa, que é fundamental, e garante a otimização de fluxo da ONG.

Na controller, este trecho de código exemplifica, o filtro, onde se for identificado que o status pós-atualização de pedido de ajuda é “APROVADO”, segue pelo fluxo que cria o beneficiário. Caso não, seria o “REJEITADO”, que apenas atualiza o pedido, e elimina a possibilidade de ser gerado um beneficiário a partir desse pedido.

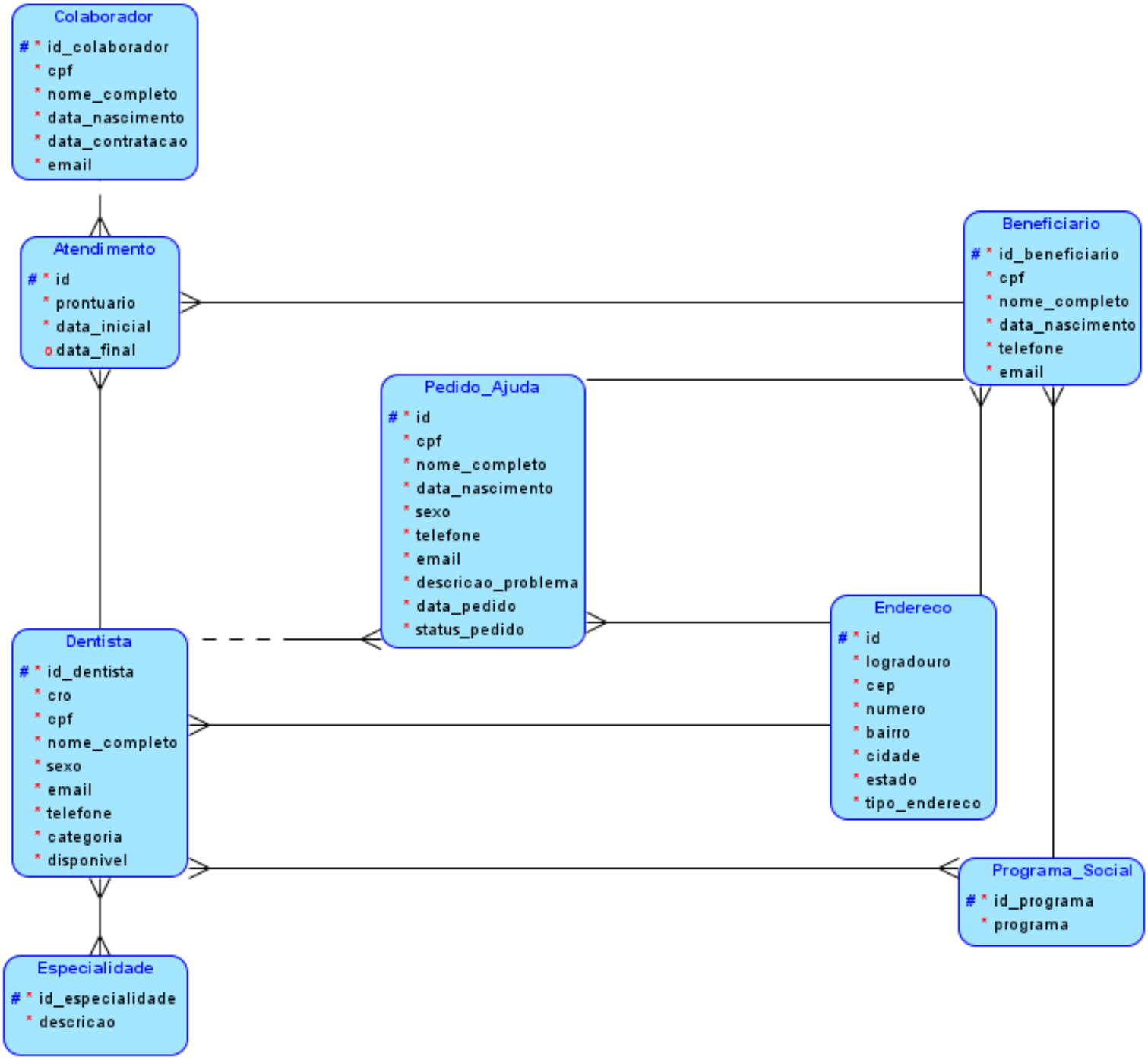
```
if(StatusPedido.APROVADO.equals(status)){
    int idPrograma = view.inputPrograma();
    bo.validarGerarBeneficiario(id, pedido, idPrograma);
}
else{
  bo.atualizar(id, pedido);
}
```

5. Método invalidarHomens -> PedidoAjudaBO

```
public static boolean invalidarHomens(LocalDate dataNasc){
        Period idade = Period.between(dataNasc, LocalDate.now());
        return idade.getYears() >= 18;
    }
```

Como a ONG atende apenas crianças e mulheres que sofreram violência sexual, caso seja identificado, que um registro de pedido de ajuda foi criado com o sexo masculino e idade igual ou maior a 18 anos, Automaticamnete o pedido já é REJEITADO. Método pega o período entre a data de nascimento e data atual, e retorna true, caso a diferença seja de 18 anos = idade.

MODELO DE ENTIDADE RELACIONAMENTO – (MER)



Modelo de Entidade Relacionamento, o MER gerado no Oracle DB, que serve de base para o nosso sistema, e exemplifica, como ficam as relações das entidades no banco de dados. As entidades do banco de dados são:

Pessoas

- Beneficiario

- Dentista
- Colaborador

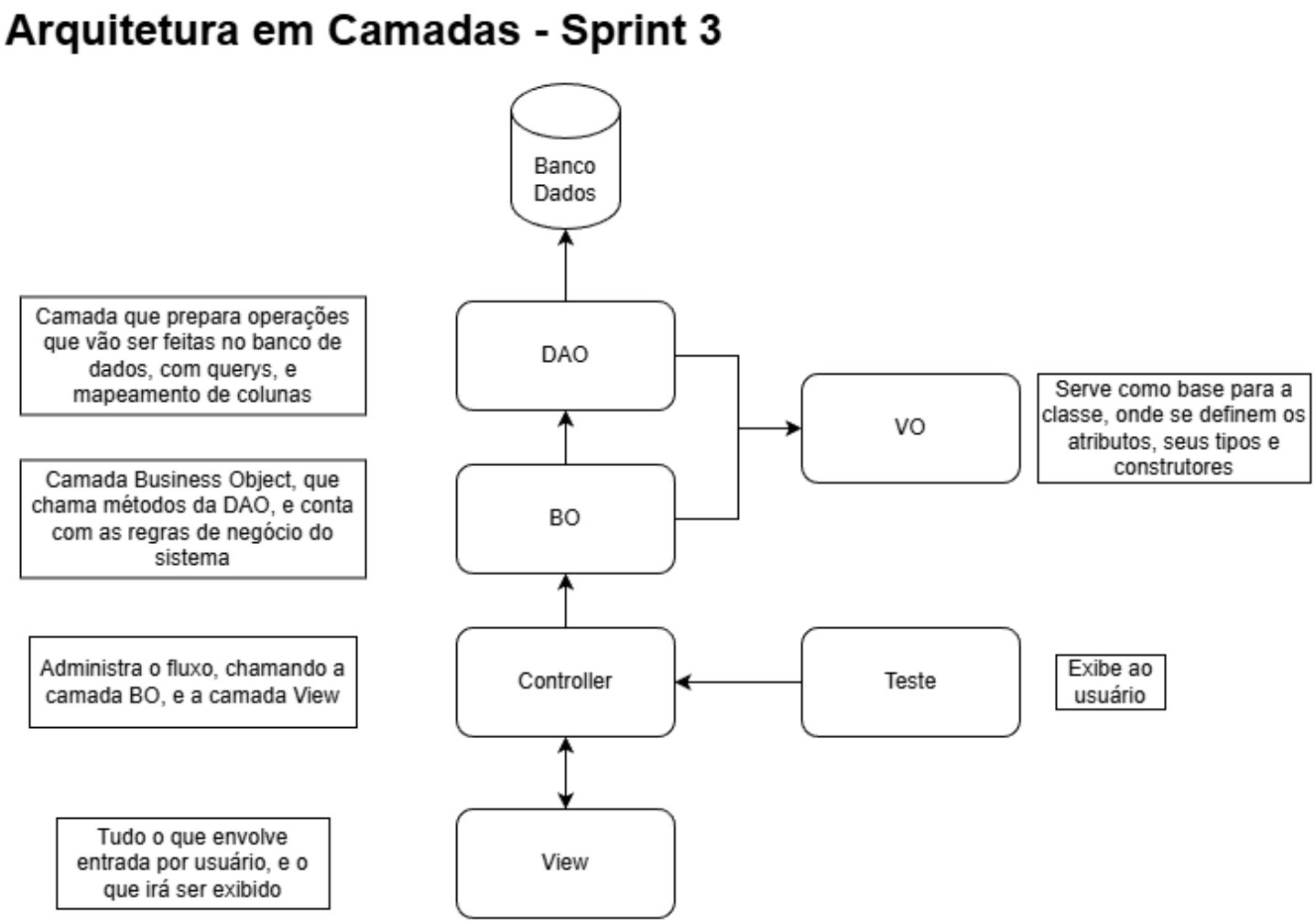
Processos

- Pedido_ajuda
- Atendimento
- Endereço

Apoio

- Especialidade
- Programa_social

ARQUITETURA DO PROJETO



Nossa arquitetura foi desenhada para evitar acoplamento. A camada de View é totalmente isolada do banco de dados, ela só conhece a Controller. As regras de

negócio ficam centralizadas em cada BO, enquanto toda a complexidade do SQL e do JDBC puro ficam nas suas respectivas DAOs. A VO, parte da Model atua como nossa estrutura de dados comum, permitindo que a informação caminhe de forma organizada em todas as camadas.

DIAGRAMAS DE CLASSES

Diagrama de Classes Simples

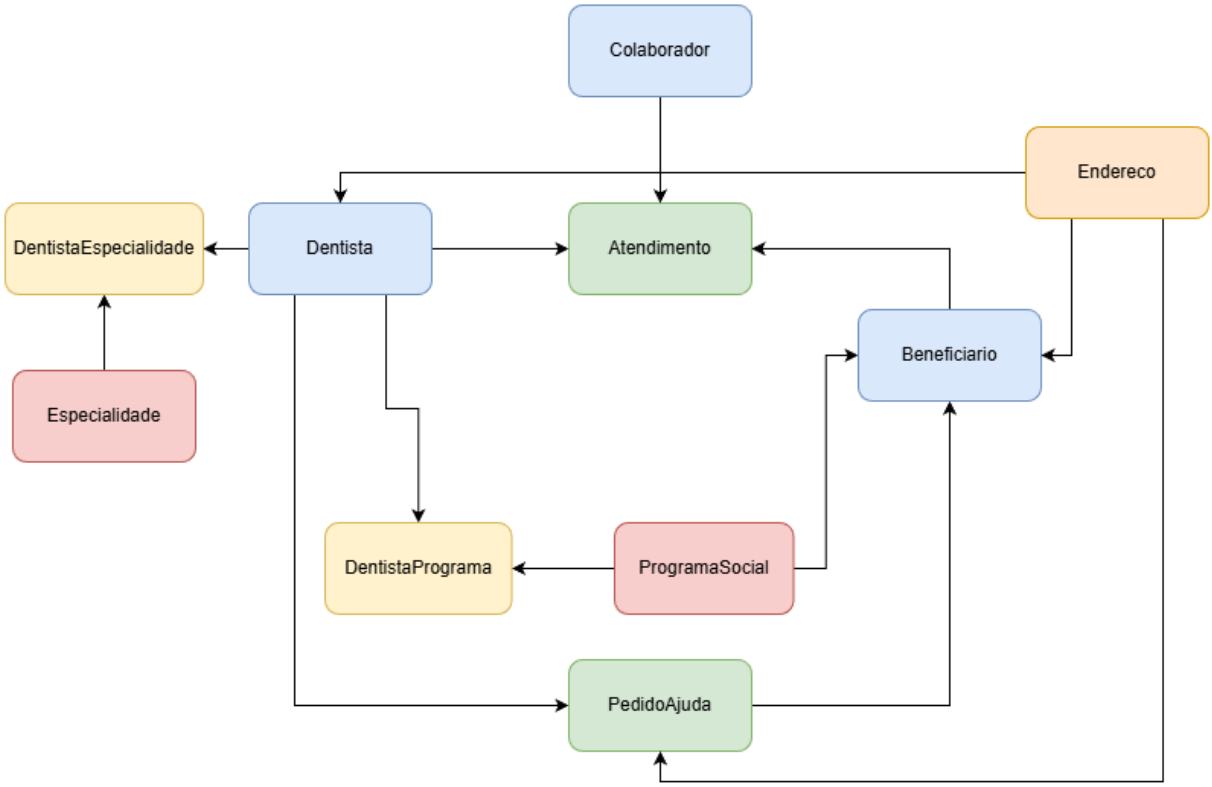


Diagrama mais simples, explicando relações das principais Classe do projeto, e como elas se ligam, refletindo o banco de dados.

Classes presentes no diagrama são:

Colaborador -> Funcionário interno da ONG

Beneficiario -> Pessoa que recebe auxílio da ONG

Dentista -> Quem presta atendimento aos beneficiários

PedidoAjuda -> Solicitação prévia, com o objetivo de receber atendimento

Atendimento -> Processo de resolução de problema do beneficiário, atendido pelo Dentista

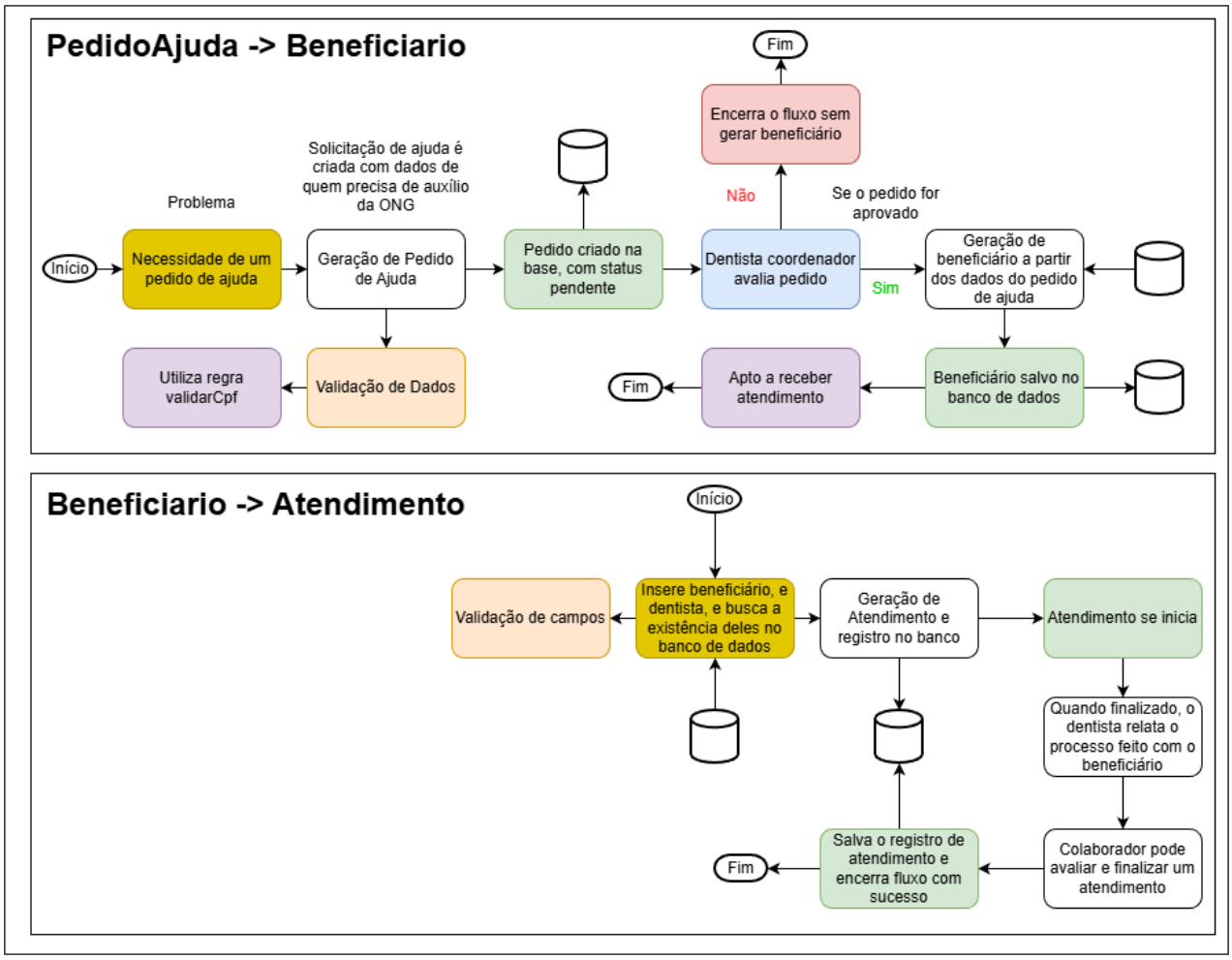
Endereco -> Local de atendimento dos dentistas/local de moradia de beneficiários.

ProgramaSocial -> Serviços da ONG, como Dentista do Bem, que atende crianças e Apolônias do Bem que atende mulheres que sofreram algum tipo de violência.

DentistaPrograma -> Representada como o público atendido pelo dentista

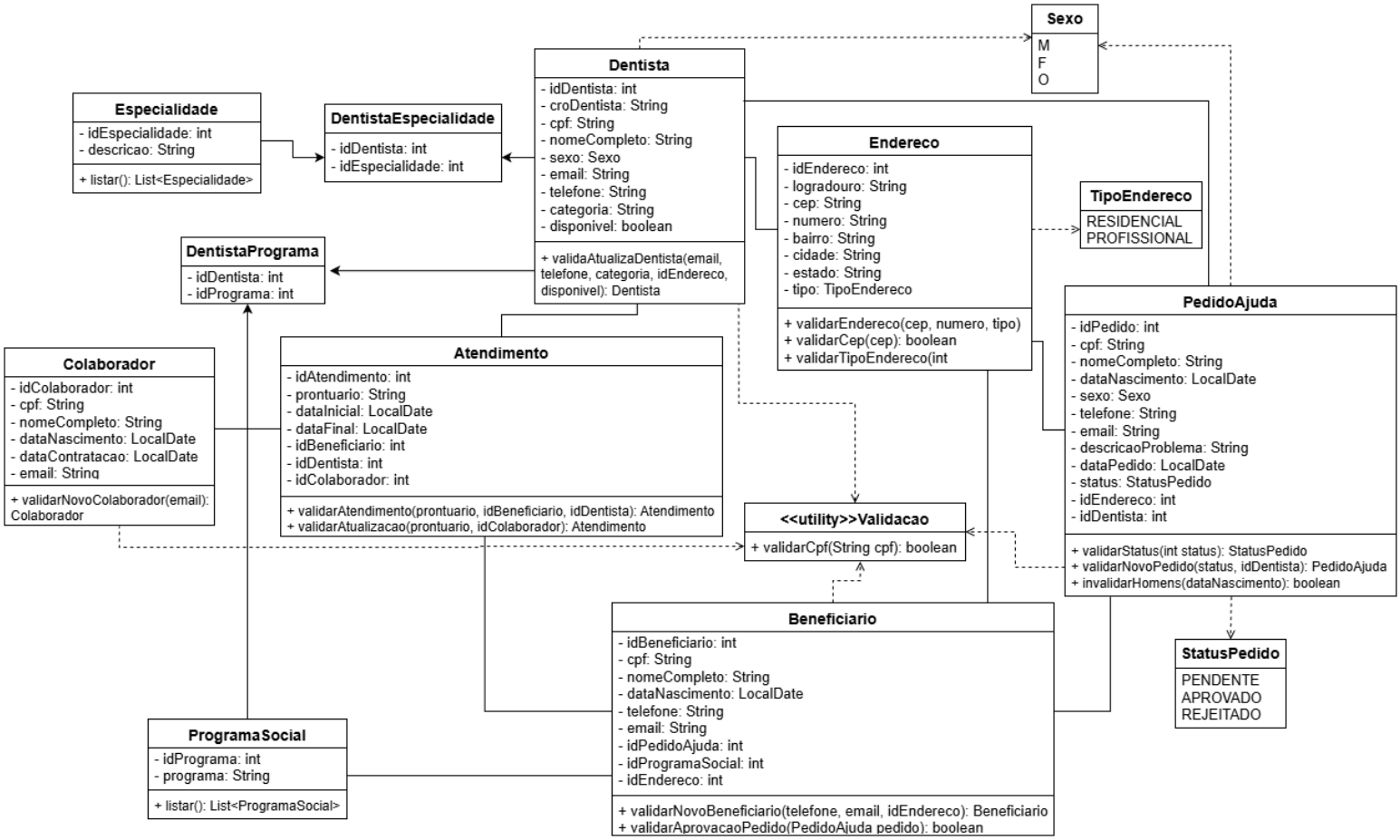
DentistaEspecialidade -> Representada como habilidades do dentista para sua atuação

Fluxo Centralizado do Sistema



Com esse diagrama, é possível mostrar, o fluxo central do sistema, e como funciona o caminho entre Pedido Ajuda -> Beneficiário -> Atendimento, que têm envolvimento das partes Dentista e Colaborador que aprovam/relatam informações dos processos.

Diagrama UML



PROCEDIMENTOS PARA EXECUÇÃO

Requisitos:

- Java JDK 21.0.0
- Oracle DB
- IDE (IntelliJ, Eclipse, NetBeans)

1. **Clonar o repositório do GitHub**
2. **Abrir o projeto em uma IDE (IntelliJ, Eclipse ou NetBeans)**
3. **Configurar o Banco Oracle:**

```
Executar o script para a criação das tabelas:
--Apagando as tabelas iniciais para não haver conflito
DROP TABLE Dentista_Especialidade CASCADE CONSTRAINTS;
DROP TABLE Dentista_Programa_Social CASCADE CONSTRAINTS;
DROP TABLE Atendimento CASCADE CONSTRAINTS;
DROP TABLE Beneficiario CASCADE CONSTRAINTS;
DROP TABLE Pedido_Ajuda CASCADE CONSTRAINTS;
DROP TABLE Colaborador CASCADE CONSTRAINTS;
DROP TABLE Dentista CASCADE CONSTRAINTS;
DROP TABLE Endereco CASCADE CONSTRAINTS;
DROP TABLE Especialidade CASCADE CONSTRAINTS;
DROP TABLE Programa_Social CASCADE CONSTRAINTS;
```

```
--Criando todas as tabelas
CREATE TABLE Atendimento (
    id_atendimento    NUMBER GENERATED ALWAYS AS IDENTITY (START WITH 1
INCREMENT BY 1) NOT NULL,
    prontuario VARCHAR2 (500 CHAR) NOT NULL,
    data_inicial      DATE NOT NULL,
    data_final        DATE NULL,
    id_beneficiario   NUMBER NOT NULL,
    id_dentista       NUMBER NOT NULL,
    id_colaborador    NUMBER NULL);
ALTER TABLE Atendimento ADD CONSTRAINT atendimento_pk PRIMARY KEY
(id_atendimento);
CREATE TABLE Beneficiario (
    id_beneficiario   NUMBER GENERATED ALWAYS AS IDENTITY (START WITH 1
INCREMENT BY 1) NOT NULL,
    cpf               CHAR (11 CHAR) NOT NULL UNIQUE,
    nome_completo     VARCHAR2 (50 CHAR) NOT NULL,
    data_nascimento   DATE NOT NULL,
    telefone          VARCHAR2 (15 CHAR) NOT NULL,
    email             VARCHAR2 (40 CHAR) NOT NULL,
    id_programa_social NUMBER NOT NULL,
    id_pedido_ajuda   NUMBER NOT NULL,
    id_endereco       NUMBER NOT NULL);
ALTER TABLE Beneficiario ADD CONSTRAINT beneficiario_pk PRIMARY KEY
(id_beneficiario);

CREATE TABLE Colaborador (
    id_colaborador    NUMBER GENERATED ALWAYS AS IDENTITY (START WITH 1
INCREMENT BY 1) NOT NULL,
    cpf               CHAR (11 CHAR) NOT NULL UNIQUE,
    nome_completo     VARCHAR2 (50 CHAR) NOT NULL,
    data_nascimento   DATE NOT NULL,
    data_contratacao  DATE NOT NULL,
    email             VARCHAR2 (40 CHAR) NOT NULL);
ALTER TABLE Colaborador ADD CONSTRAINT colaborador_pk PRIMARY KEY
(id_colaborador);

CREATE TABLE Dentista (
    id_dentista   NUMBER GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT
BY 1) NOT NULL,
    cro           VARCHAR2 (10 CHAR) NOT NULL UNIQUE,
    cpf           CHAR (11 CHAR) NOT NULL UNIQUE,
    nome_completo VARCHAR2 (50 CHAR) NOT NULL,
    sexo          CHAR (1 CHAR) NOT NULL CHECK(sexo IN ('M', 'F', 'O')),
    email         VARCHAR2 (40 CHAR) NOT NULL,
    telefone      VARCHAR2 (15 CHAR) NOT NULL,
    categoria     VARCHAR2 (11 CHAR) NOT NULL CHECK(categoria IN
('COORDENADOR', 'CLINICO')),
    id_endereco   NUMBER NOT NULL,
    disponivel    CHAR (1 CHAR) NOT NULL CHECK(disponivel IN ('S', 'N')));
ALTER TABLE Dentista ADD CONSTRAINT dentista_pk PRIMARY KEY (id_dentista);

CREATE TABLE Dentista_Especialidade (
    id_especialidade NUMBER NOT NULL,
    id_dentista      NUMBER NOT NULL);
ALTER TABLE Dentista_Especialidade ADD CONSTRAINT dent_espec_pk PRIMARY KEY
(id_especialidade, id_dentista);

CREATE TABLE Dentista_Programa_Social (
    id_dentista NUMBER NOT NULL,
    id_programa NUMBER NOT NULL);
ALTER TABLE Dentista_Programa_Social ADD CONSTRAINT dentista_programa_pk
PRIMARY KEY (id_dentista, id_programa);

CREATE TABLE Endereco (
```

```
    id_endereco   NUMBER GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT
BY 1) NOT NULL,
    logradouro    VARCHAR2 (50 CHAR) NOT NULL,
    cep           CHAR (8 CHAR) NOT NULL,
    numero        VARCHAR2 (6 CHAR) NOT NULL,
    bairro        VARCHAR2 (40 CHAR) NOT NULL,
    cidade        VARCHAR2 (30 CHAR) NOT NULL,
    estado        VARCHAR2 (30 CHAR) NOT NULL,
    tipo_endereco VARCHAR2 (12 CHAR) NOT NULL CHECK(tipo_endereco IN
('RESIDENCIAL', 'PROFISSIONAL')));
ALTER TABLE Endereco ADD CONSTRAINT endereco_pk PRIMARY KEY (id_endereco);

CREATE TABLE Especialidade (
    id_especialidade NUMBER GENERATED ALWAYS AS IDENTITY (START WITH 1
INCREMENT BY 1) NOT NULL,
    descricao        VARCHAR2 (25 CHAR) NOT NULL);
ALTER TABLE Especialidade ADD CONSTRAINT especialidade_pk PRIMARY KEY
(id_especialidade);

CREATE TABLE Pedido_Ajuda (
    id_pedido          NUMBER GENERATED ALWAYS AS IDENTITY (START WITH 1
INCREMENT BY 1) NOT NULL,
    cpf                CHAR (11 CHAR) NOT NULL UNIQUE,
    nome_completo      VARCHAR2 (50 CHAR) NOT NULL,
    data_nascimento    DATE NOT NULL,
    sexo               CHAR (1 CHAR) NOT NULL CHECK(sexo IN ('M', 'F', 'O')),
    telefone           VARCHAR2 (15 CHAR) NOT NULL,
    email              VARCHAR2 (40 CHAR) NOT NULL,
    descricao_problema VARCHAR2 (300 CHAR) NOT NULL,
    data_pedido        DATE NOT NULL,
    status_pedido      VARCHAR2 (20 CHAR) NOT NULL CHECK(status_pedido IN
('PENDENTE', 'APROVADO', 'REJEITADO')),
    id_dentista        NUMBER NULL,
    id_endereco        NUMBER NOT NULL);
ALTER TABLE Pedido_Ajuda ADD CONSTRAINT pedido_ajuda_pk PRIMARY KEY
(id_pedido);

CREATE TABLE Programa_Social (
    id_programa NUMBER NOT NULL,
    programa    VARCHAR2 (30 CHAR) NOT NULL);
ALTER TABLE Programa_Social ADD CONSTRAINT programa_social_pk PRIMARY KEY
(id_programa);
ALTER TABLE Atendimento ADD CONSTRAINT atendimento_beneficiario_fk FOREIGN KEY
(id_beneficiario) REFERENCES Beneficiario (id_beneficiario);
ALTER TABLE Atendimento ADD CONSTRAINT atendimento_colaborador_fk FOREIGN KEY
(id_colaborador) REFERENCES Colaborador (id_colaborador);
ALTER TABLE Atendimento ADD CONSTRAINT atendimento_dentista_fk FOREIGN KEY
(id_dentista) REFERENCES Dentista (id_dentista);
ALTER TABLE Beneficiario ADD CONSTRAINT beneficiario_endereco_fk FOREIGN KEY
(id_endereco) REFERENCES Endereco (id_endereco);
ALTER TABLE Beneficiario ADD CONSTRAINT beneficiario_pedido_fk FOREIGN KEY
(id_pedido_ajuda) REFERENCES Pedido_Ajuda (id_pedido);
ALTER TABLE Beneficiario ADD CONSTRAINT beneficiario_programa_fk FOREIGN KEY
(id_programa_social) REFERENCES Programa_Social (id_programa);
ALTER TABLE Dentista_Especialidade ADD CONSTRAINT dent_espec_den_fk FOREIGN KEY
(id_dentista) REFERENCES Dentista (id_dentista);
ALTER TABLE Dentista_Especialidade ADD CONSTRAINT dent_espec_espec_fk FOREIGN
KEY (id_especialidade) REFERENCES Especialidade (id_especialidade);
ALTER TABLE Dentista ADD CONSTRAINT dentista_end_fk FOREIGN KEY (id_endereco)
REFERENCES Endereco (id_endereco);
ALTER TABLE Dentista_Programa_Social ADD CONSTRAINT dentista_programa_dent_fk
FOREIGN KEY (id_dentista) REFERENCES Dentista (id_dentista);
ALTER TABLE Dentista_Programa_Social ADD CONSTRAINT dentista_programa_prog_fk
FOREIGN KEY (id_programa) REFERENCES Programa_Social (id_programa);
```

```
ALTER TABLE Pedido_Ajuda ADD CONSTRAINT pedido_ajuda_dentista_fk FOREIGN KEY
(id_dentista) REFERENCES Dentista (id_dentista);
ALTER TABLE Pedido_Ajuda ADD CONSTRAINT pedido_ajuda_endereco_fk FOREIGN KEY
(id_endereco) REFERENCES Endereco (id_endereco);
```

4. **Adicionar dependências:**

- JDBC Oracle
- Gson

5. **Inserir credenciais no arquivo Conexao.Java:**

- Corrigir trecho de código das linhas 19 e 20, inserindo corretamente as credencias do banco de dados, usuário e senha Oracle

6. **Executar por classe de teste**

O sistema é executado via classes de teste com método main, simulando funcionalidades no console.

EnderecoTeste
DentistaTeste
ColaboradorTeste
BeneficiarioTeste
PedidoTeste
AtendimentoTeste
EspecialidadeTeste
ProgramaSocialTeste

7. **Utilizar métodos criados para simular o uso do sistema**

DEMONSTRAÇÃO DE FUNCIONAMENTO DO FLUXO

Criar Pedido de Ajuda

```
Retorno - adicionarPedido()

Criando pedido de ajuda:

Digite o CPF:
99900055533

Digite o nome:
João da Silva Coutinho

Insira a data de nascimento (dd/MM/yyyy):
09/10/2010

Selecione o sexo do solicitante:
1. Masculino
2. Feminino
3. Outros
1

Telefone do solicitante:
11990000322

Email do solicitante:
coutinho@coldmail.com

Descrição completa do pedido de ajuda:
garoto de 16 anos, com diversos buracos nos dentes, chegando a afetar a mordida.

Insira o ID do endereço do pedido:
9

Pedido de ajuda criado com sucesso!
```

Exemplo de como funciona a criação de um pedido de ajuda no console, quais campos são solicitados.

Listagem de pedidos de ajuda

```
Retorno - listarTodos()

Exibindo todos os pedidos de ajuda:

1 - Solicitante: Guilherme Amado - Cpf: 11122233344
    Descrição: Atacante risonho, precisa de correção, com o clínico geral.
    Data de Nascimento: 13/05/1995 - Sexo: M
    Telefone: 11970709999 - email: amado@gmail.com
    Data do Pedido: 04/04/2026 - status: PENDENTE

2 - Solicitante: Fernanda Lima Santos - Cpf: 22233344455
    Descrição: Dor aguda no dente molar esquerdo, sensibilidade ao frio e calor.
    Data de Nascimento: 10/08/1990 - Sexo: F
    Telefone: 11988887777 - email: fernanda.lima@email.com
    Data do Pedido: 05/04/2026 - status: PENDENTE

3 - Solicitante: Ricardo Oliveira Souza - Cpf: 33344455566
    Descrição: Gengiva sangrando ao escovar os dentes há 3 dias.
    Data de Nascimento: 22/03/1985 - Sexo: M
    Telefone: 11999996666 - email: ricardo.souza@email.com
    Data do Pedido: 05/04/2026 - status: PENDENTE

4 - Solicitante: Carla Mendes Rocha - Cpf: 44455566677
    Descrição: Quebrou um pedaço do dente da frente ao comer.
    Data de Nascimento: 17/12/1992 - Sexo: F
    Telefone: 11977775555 - email: carla.rocha@email.com
    Data do Pedido: 04/04/2026 - status: PENDENTE

5 - Solicitante: Paulo Henrique Dias - Cpf: 55566677788
    Descrição: Dor forte ao mastigar, suspeita de cárie profunda.
    Data de Nascimento: 05/07/1988 - Sexo: M
    Telefone: 11966664444 - email: paulo.dias@email.com
    Data do Pedido: 04/04/2026 - status: PENDENTE

6 - Solicitante: Juliana Costa Almeida - Cpf: 66677788899
    Descrição: Dente do siso nascendo, causando dor e inchaço na gengiva.
    Data de Nascimento: 30/01/1995 - Sexo: F
    Telefone: 11955553333 - email: juliana.almeida@email.com
    Data do Pedido: 03/04/2026 - status: PENDENTE

21 - Solicitante: Lucas Silva Mendes - Cpf: 12345678909
    Descrição: Dor de dente constante ao mastigar, sensível a alimentos frios.
    Data de Nascimento: 15/08/2010 - Sexo: M
    Telefone: 11988887777 - email: lucas.mendes@email.com
    Data do Pedido: 06/04/2026 - status: APROVADO

61 - Solicitante: João da Silva Coutinho - Cpf: 99900055533
    Descrição: garoto de 16 anos, com diversos buracos nos dentes, chegando a afetar
a mordida.
    Data de Nascimento: 09/10/2010 - Sexo: M
    Telefone: 11990000322 - email: coutinho@coldmail.com
    Data do Pedido: 09/04/2026 - status: PENDENTE
```

Listando todos os pedidos de ajuda

No método de listagem foi possível observar a listagem de todos os pedidos de ajuda corretamente, todos formatados e exibindo as informações dos pedidos.

Atualizando pedido de ajuda

```
Retorno - atualizar()

//Dados do pedido que eu queria atualizar
61 - Solicitante: João da Silva Coutinho - Cpf: 99900055533
     Descrição: garoto de 16 anos, com diversos buracos nos dentes, chegando a
afetar a mordida.
     Data de Nascimento: 09/10/2010 - Sexo: M
     Telefone: 11990000322 - email: coutinho@coldmail.com
     Data do Pedido: 09/04/2026 - status: PENDENTE

Insira o ID:
61

Atualizando pedido de ajuda:

Selecione o novo status do pedido:
1. APROVADO
2. REJEITADO
1

Insira o ID do dentista responsável:
25

Esse beneficiário(a) fará parte de qual programa da ONG?
1. Dentista do Bem
2. Apolônias do Bem
1

Pedido atualizado com sucesso!!!

21 - Solicitante: Lucas Silva Mendes - Cpf: 12345678909
     Descrição: Dor de dente constante ao mastigar, sensível a alimentos frios.
     Data de Nascimento: 15/08/2010 - Sexo: M
     Telefone: 11988887777 - email: lucas.mendes@email.com
     Data do Pedido: 06/04/2026 - status: APROVADO
```

Atualizando pedido de ajuda

Com esse método foi possível atualizar um pedido específico, utilizando a busca pelo id do pedido de ajuda, e ao notar que o status se tornou “APROVADO”, pude recuperar na base, a verificação se realmente a criação de beneficiário havia sido feita e inserida corretamente. Com a query sql:

- select * from beneficiario where cpf = '12345678909';

Planilha | Query Builder

```
select * from beneficiario where cpf  = '12345678909'
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,003 segundos

	ID_BENEFICIARIO	CPF	NOME_COMPLETO	DATA_NASCIMENTO	TELEFONE	EMAIL	ID_PROGRAMA_SOCIAL	ID_PEDIDO_AJUDA	ID_ENDERECO
1	21	12345678909	Lucas Silva Mendes	15/08/10	11988887777	lucas.mendes@email.com	1	21	9

Verificando usuário criado automaticamente

Correspondente ao mesmo que buscar o beneficiário com o cpf que inserimos acima, foi possível perceber,que já consta na tabela, o novo beneficiário com todos os dados inseridos pela classe de pedido de ajuda.

Criando Atendimento pós beneficiário criado

```
Retorno - criarAtendimento()

Insira o prontuário do atendimento:
Início do tratamento do Coutinho. Mordida afetada por cáries.

Digite o ID do beneficiário que irá receber o atendimento:
3

Digite o ID do dentista que irá fornecer o atendimento:
7

Atendimento criado com sucesso!!!
```

Criar Atendimento

Procedimento é bem simples, apenas inserir o prontuário, que estará disponível para atualização posteriormente, id do beneficiário e do dentista.

```
Retorno - listandoatendimento()

Digite o id do atendimento:
7

Insira o prontuário do atendimento:
Fim do tratamento do coutinho, tudo resolvido, apos colocar aparelho e tratar bem
com remedios x

Digite o ID do colaborador que está finalizando o atendimento:
5

Atendimento atualizado com sucesso!!!

7 -  Prontuário: Fim do tratamento do coutinho, tudo resolvido, apos colocar
aparelho e tratar bem com remedios x
     Data inicial: 10/04/2026 - Data Final: 10/04/2026

     Id do beneficiário atendido: 3
     Id do dentista que atende: 7
     Id colaborador responsável: 5
```

Atualizando e finalizando o atendimento

Aqui eu unifiquei em uma única imagem, mas basicamente, o trecho de cima, é referente à atualização com o fechamento do atendimento, que recebe a data final, capturando do sistema do usuário, o id do colaborador que está fechando e um detalhamento completo sobre como foi. Abaixo ele exibe na tela para mostrar como fica um atendimento finalizado, com todas as informações de beneficiário, dentista atendente e o colaborador relator.

EXECUÇÃO DO MAIN COM FLUXO CENTRAL

Criação de Pedido de ajuda

```
Criando pedido de ajuda:

Digite o CPF:
12562289207

Digite o nome:
Teste do Main

Insira a data de nascimento (dd/MM/yyyy):
19/10/2012

Selecione o sexo do solicitante:
1. Masculino
2. Feminino
3. Outros
1
```

```
Run  Main
3. Outros
1

Telefone do solicitante:
11998929291

Email do solicitante:
teste@gmail.com

Descrição completa do pedido de ajuda:
Testando inserção de pedido de ajuda, que virará beneficiário e depois um atendimento.

Insira o ID do endereço do pedido:
7

Pedido de ajuda criado com sucesso!
```

Exibindo Pedido criado

```
      Data de Nascimento: 20/01/2015 - Sexo: M
      Telefone: 11973645211 - email: pedro.h@email.com
      Data do Pedido: 10/04/2026 - status: PENDENTE

7 - Solicitante: Sofia Castro - Cpf: 33377766644
      Descrição: Limpeza e prevenção infantil.
      Data de Nascimento: 30/08/2018 - Sexo: F
      Telefone: 11964532100 - email: sofia.c@email.com
      Data do Pedido: 10/04/2026 - status: PENDENTE

21 - Solicitante: Teste do Main - Cpf: 12562289207
      Descrição: Testando inserção de pedido de ajuda, que virará beneficiário e depois um atendimento.
      Data de Nascimento: 19/10/2012 - Sexo: M
      Telefone: 11998929291 - email: teste@gmail.com
      Data do Pedido: 10/04/2026 - status: PENDENTE
```

Aprovando pedido

```
Run  Main
21 - Solicitante: Teste do Main - Cpf: 12562289207
      Descrição: Testando inserção de pedido de ajuda, que virará beneficiário e depois um atendimento.
      Data de Nascimento: 19/10/2012 - Sexo: M
      Telefone: 11998929291 - email: teste@gmail.com
      Data do Pedido: 10/04/2026 - status: PENDENTE

Insira o ID:
21

Atualizando pedido de ajuda:

Selecione o novo status do pedido:
1. APROVADO
2. REJEITADO
1

Insira o ID do dentista responsável:
4

Esse beneficiário(a) fará parte de qual programa da ONG?
1. Dentista do Bem
2. Apolônias do Bem
1

Pedido atualizado com sucesso!!!
```

Verificando se virou beneficiário corretamente

```
Exibindo todos os beneficiários:
 1 - Carla Mendes Rocha - Cpf: 44445556677
     Data de nascimento: 17/12/1992 - Telefone: 11999997070 - email: carlamendesrocha@coldmail.com
     Pedido de ajuda: 4, Programa Social: 2
 2 - Lucas Silva Mendes - Cpf: 12345678909
     Data de nascimento: 15/08/2010 - Telefone: 11988887777 - email: lucas.mendes@email.com
     Pedido de ajuda: 2, Programa Social: 1
 3 - João da Silva Coutinho - Cpf: 99900055533
     Data de nascimento: 09/10/2010 - Telefone: 11990000322 - email: coutinho@coldmail.com
     Pedido de ajuda: 3, Programa Social: 1
21 - Teste do Main - Cpf: 12562289207
     Data de nascimento: 19/10/2012 - Telefone: 11998929291 - email: teste@gmail.com
     Pedido de ajuda: 21, Programa Social: 1
```

Iniciando Atendimento

```
  21 - Teste do Main - Cpf: 12562289207
      Data de nascimento: 19/10/2012 - Telefone: 11998929291 - email: teste@gmail.com
      Pedido de ajuda: 21, Programa Social: 1

Criando atendimento:

Insira o prontuário do atendimento:
TESTANDO INICIO DE ATENDIMENTO DO TESTE DO MAIN, PROBLEMA DENTAL, ENVOLVE BENEFICIÁRIO DO PROGRAMA DENTISTA DO BEM E PRECISA DE
  UM ATENDIMENTO URGENTE

Digite o ID do beneficiário que irá receber o atendimento:
21

Digite o ID do dentista que irá fornecer o atendimento:
6

Atendimento criado com sucesso!!!
```

Verificando atendimento criado

```
Digite o CPF:
12562289207

Atendimento encontrado:

21 -  Prontuário: TESTANDO INICIO DE ATENDIMENTO DO TESTE DO MAIN, PROBLEMA DENTAL, ENVOLVE BENEFICIÁRIO DO PROGRAMA
 DENTISTA DO BEM E PRECISA DE UM ATENDIMENTO URGENTE
     Data inicial: 10/04/2026 - Data Final: Em aberto

     Id do beneficiário atendido: 21
     Id do dentista que atende: 6
     Id colaborador responsável: Colaborador relator pendente
```

Atualizando atendimento

```
Digite o id do atendimento:
21

Insira o prontuário do atendimento:
FINALIZANDO O ATENDIMENTO TESTES, PARA VALIDAR O FLUXO NO CONSOLE, NA CLASSE MAIN.TUDO FEITO 100%

Digite o ID do colaborador que está finalizando o atendimento:
3

Atendimento atualizado com sucesso!!!
```

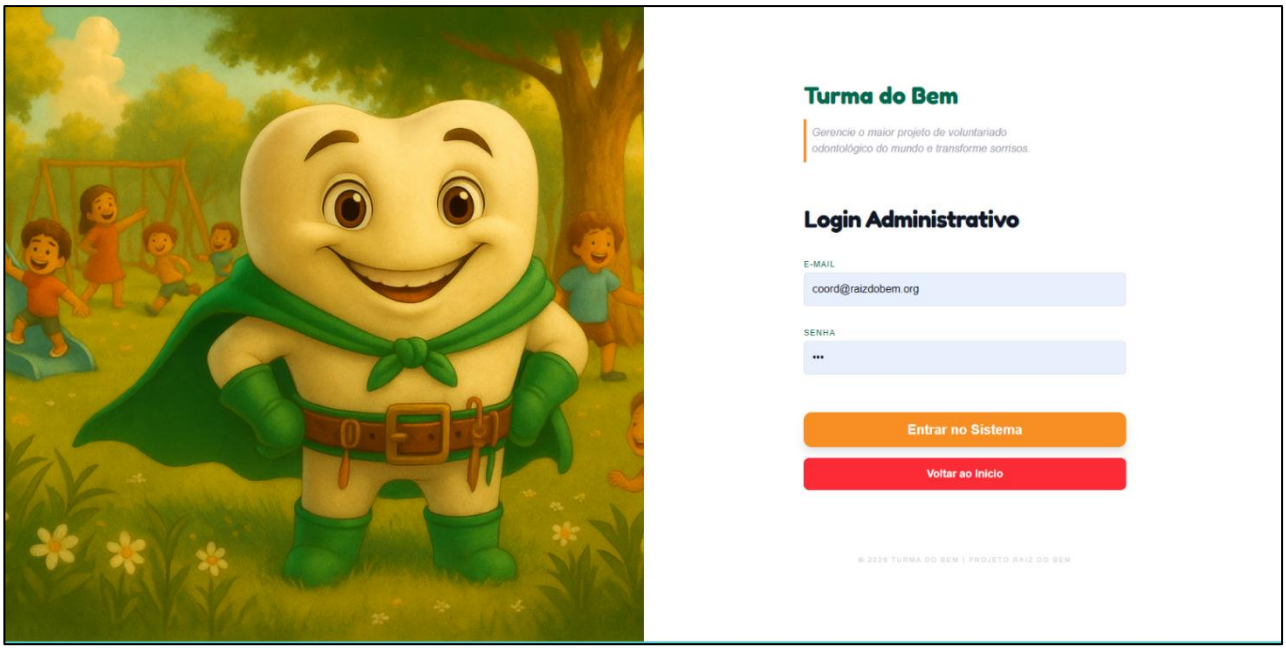
Exibindo atendimento

```
Digite o CPF:
12562289207

Atendimento encontrado:

21 -  Prontuário: FINALIZANDO O ATENDIMENTO TESTES, PARA VALIDAR O FLUXO NO CONSOLE, NA CLASSE MAIN.TUDO FEITO 100%
     Data inicial: 10/04/2026 - Data Final: 10/04/2026

     Id do beneficiário atendido: 21
     Id do dentista que atende: 6
     Id colaborador responsável: 3

Process finished with exit code 0
```

REFERÊNCIAS E LINKS RELEVANTES

Repositório do projeto no GitHub: [RaizDoBem-Backend-Java]
Site da nossa iniciativa – Raiz do Bem: [Raiz do Bem]

Caso deseje fazer login no nosso site e verificar o andamento:



Email: coord@raizdobem.org – Usuário Teste
Senha: 123

CONCLUSÃO

O projeto desenvolvido atende integralmente aos requisitos da Sprint 3, apresentando um sistema 100% funcional, com operações completas de CRUD e fluxo validado de ponta a ponta.

A aplicação foi estruturada em arquitetura em camadas (View, Controller, BO, DAO e VO), promovendo clara separação de responsabilidades, organização e facilidade de manutenção. Mesmo utilizando Java puro com JDBC, o projeto incorpora conceitos do padrão MVC e boas práticas alinhadas ao Domain-Driven Design.

Como diferencial, destaca-se a implementação de regras de negócio relevantes, como validação de CPF e CEP com uso de expressões regulares, além da integração com a API ViaCEP, que automatiza o preenchimento de dados e melhora a experiência do usuário.

O fluxo central - desde o pedido de ajuda até o atendimento do beneficiário - foi completamente implementado e validado, garantindo consistência, rastreabilidade e integridade das informações.

O resultado demonstra o cumprimento dos requisitos propostos, e a aplicação sólida de conceitos com evolução significativa na construção de sistemas organizados, funcionais e preparados para escalabilidade.